

Organisation des données côté serveur

Projet : SAE BUT2 S3 2026-2027 **Thème :** Langages de programmation **Responsables :** Mai + Albine **Partie :** Serveur C + Sockets

Table des matières

- 1. Objectif
- 2. Structures C complètes
 - 2.1 Structure Carte
 - 2.2 Structure Utilisateur
 - 2.3 Structure Recommandation
 - 2.4 Structure Battle
 - 2.5 Structure Vote
 - 2.6 Structure ActionCarte
 - 2.7 Structure Block
 - 2.8 Structure Blockchain
- 3. Conteneurs globaux
- 4. Synchronisation
- 5. Communication avec le client
 - 5.1 Réception des requêtes
 - 5.2 Parsing JSON
 - 5.3 Exemple de parsing complet
 - 5.4 Exemple de création de réponse
 - 5.5 Mapping JSON ↔ C
- 6. Lien avec la blockchain
- 7. Lien avec PostgreSQL
 - 7.1 Principe
 - 7.2 Mapping structures C vers tables SQL
 - 7.3 Fonctions de connexion
 - 7.4 Structure de la connexion
 - 7.5 Exemple d'insertion en BDD
- 8. Prototypes de fonctions
- 9. Gestion des erreurs
- 10. Cycle de vie d'une donnée
- 11. Architecture des modules
- 12. Récapitulatif
- 13. Validation

1. Objectif

Ce document décrit l'organisation complète des données côté serveur C.

Il présente :

- Les structures C avec leurs types exacts
 - Les conteneurs globaux
 - La synchronisation (mutex)
 - La communication avec le client (JSON)
 - Le lien avec la blockchain
 - Le lien avec PostgreSQL (libpq)
 - Les prototypes de fonctions
 - La gestion des erreurs
 - L'architecture des modules
-

2. Structures C complètes

2.1 Structure Carte

La structure Carte représente une carte langage.

```
typedef struct {  
    int id;  
    char title[100];  
    char paradigm[30];  
    char description[500];  
    char typing[20];  
    char difficulty[20];  
    int year_created;  
    char image_url[500];  
    int creator_id;  
    int owner_id;  
    char status[20];  
    int value;  
    int is_authenticated;  
    int win_count;  
    long created_at;  
} Carte;
```

Taille totale : environ 1200 octets

2.2 Structure Utilisateur

La structure Utilisateur représente un client connecté.

```
typedef struct {  
    int id;  
    char username[50];  
    int socket_fd;  
    pthread_t thread_id;  
    int connected;  
    int reco_count;
```

```
    int legitimacy_count;  
} Utilisateur;
```

Taille totale : environ 100 octets

2.3 Structure Recommandation

La structure Recommandation représente une recommandation.

```
typedef struct {  
    int id;  
    int user_id;  
    int card_id;  
    int active;  
    long timestamp;  
} Recommandation;
```

Taille totale : environ 20 octets

2.4 Structure Battle

La structure Battle représente une battle.

```
typedef struct {  
    int id;  
    int card1_id;  
    int card2_id;  
    int winner_id;  
    int status;  
    int card1_votes;  
    int card2_votes;  
    long started_at;  
    long ended_at;  
} Battle;
```

Taille totale : environ 40 octets

2.5 Structure Vote

La structure Vote représente un vote dans une battle.

```
typedef struct {  
    int id;  
    int battle_id;  
    int user_id;  
    int card_id;
```

```
    int weight;  
} Vote;
```

Taille totale : environ 20 octets

2.6 Structure ActionCarte

La structure ActionCarte représente une légitimation, revendication ou authentification.

```
typedef struct {  
    int id;  
    int card_id;  
    int user_id;  
    int type;  
    int status;  
    long requested_at;  
    long granted_at;  
} ActionCarte;
```

Taille totale : environ 30 octets

2.7 Structure Block

La structure Block représente un bloc de la blockchain.

```
typedef struct {  
    int id;  
    long timestamp;  
    char data[1024];  
    char prev_hash[65];  
    int nonce;  
    char hash[65];  
} Block;
```

Taille totale : environ 1200 octets

2.8 Structure Blockchain

La structure Blockchain représente la blockchain complète.

```
typedef struct {  
    Block *blocks;  
    int size;  
    int capacity;  
    int difficulty;
```

```
pthread_mutex_t mutex;  
} Blockchain;
```

Taille totale : environ 50 octets

3. Conteneurs globaux

3.1 Liste des cartes

```
Carte *liste_cartes = NULL;  
int nb_cartes = 0;  
int capacite_cartes = 0;
```

3.2 Liste des utilisateurs

```
Utilisateur *liste_utilisateurs = NULL;  
int nb_utilisateurs = 0;  
int capacite_utilisateurs = 0;
```

3.3 Liste des battles

```
Battle *liste_battles = NULL;  
int nb_battles = 0;  
int capacite_battles = 0;
```

3.4 Blockchain globale

```
Blockchain blockchain;
```

3.5 Opérations sur les listes

```
int add_carte(Carte *carte);  
int add_utilisateur(Utilisateur *user);  
int add_battle(Battle *battle);  
  
Carte* find_carte(int card_id);  
Utilisateur* find_utilisateur(int user_id);  
Battle* find_battle(int battle_id);
```

```
int remove_utilisateur(int user_id);  
int remove_battle(int battle_id);
```

4. Synchronisation

4.1 Mutex

Trois mutex protègent les listes.

```
pthread_mutex_t mutex_cartes = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_t mutex_utilisateurs = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_t mutex_battles = PTHREAD_MUTEX_INITIALIZER;
```

La blockchain a son propre mutex.

```
pthread_mutex_t mutex_blockchain = PTHREAD_MUTEX_INITIALIZER;
```

4.2 Utilisation

Chaque section critique utilise pthread_mutex_lock et pthread_mutex_unlock.

```
pthread_mutex_lock(&mutex_cartes);  
/* Section critique */  
pthread_mutex_unlock(&mutex_cartes);
```

4.3 Règles de synchronisation

- Un thread ne doit jamais bloquer plus longtemps que nécessaire.
- Un thread ne doit jamais acquérir deux mutex en même temps.
- Toujours libérer le mutex après utilisation.

5. Communication avec le client

5.1 Réception des requêtes

Le serveur reçoit les requêtes via le socket TCP.

Chaque requête est une ligne JSON terminée par un saut de ligne.

```
char buffer[4096];  
int valread = read(client_fd, buffer, sizeof(buffer) - 1);
```

5.2 Parsing JSON

Le serveur utilise la bibliothèque cJSON pour parser le JSON.

```
#include <cjson/cJSON.h>

cJSON *root = cJSON_Parse(buffer);
if (root == NULL) {
    send_error(client_fd, "INVALID_DATA", "JSON invalide");
    return;
}

cJSON *action = cJSON_GetObjectItem(root, "action");
cJSON *payload = cJSON_GetObjectItem(root, "payload");
```

5.3 Exemple de parsing complet

Exemple pour CREATE_CARD.

```
cJSON *root = cJSON_Parse(buffer);
cJSON *action = cJSON_GetObjectItem(root, "action");

if (strcmp(action->valuestring, "CREATE_CARD") == 0) {
    cJSON *payload = cJSON_GetObjectItem(root, "payload");

    cJSON *title = cJSON_GetObjectItem(payload, "title");
    cJSON *paradigm = cJSON_GetObjectItem(payload, "paradigm");
    cJSON *description = cJSON_GetObjectItem(payload, "description");
    cJSON *typing = cJSON_GetObjectItem(payload, "typing");
    cJSON *difficulty = cJSON_GetObjectItem(payload, "difficulty");
    cJSON *year = cJSON_GetObjectItem(payload, "yearCreated");

    Carte *carte = create_card(
        title->valuestring,
        paradigm->valuestring,
        description->valuestring,
        typing->valuestring,
        difficulty->valuestring,
        year->valueint,
        user_id
    );

    // Creer la reponse
    cJSON *response = cJSON_CreateObject();
    cJSON_AddStringToObject(response, "status", "OK");
    cJSON_AddStringToObject(response, "action", "CARD_CREATED");

    cJSON *data = cJSON_CreateObject();
    cJSON_AddNumberToObject(data, "cardId", carte->id);
    cJSON_AddItemToObject(response, "data", data);
```

```
char *response_str = cJSON_PrintUnformatted(response);
send(client_fd, response_str, strlen(response_str), 0);
send(client_fd, "\n", 1, 0);

free(response_str);
cJSON_Delete(response);
}

cJSON_Delete(root);
```

5.4 Exemple de création de réponse

Exemple pour une erreur.

```
void send_error(int client_fd, char *code, char *message) {
    cJSON *response = cJSON_CreateObject();
    cJSON_AddStringToObject(response, "status", "ERROR");
    cJSON_AddStringToObject(response, "code", code);
    cJSON_AddStringToObject(response, "message", message);

    char *json_str = cJSON_PrintUnformatted(response);
    send(client_fd, json_str, strlen(json_str), 0);
    send(client_fd, "\n", 1, 0);

    free(json_str);
    cJSON_Delete(response);
}
```

5.5 Mapping JSON ↔ C

JSON	C	Type
cardId	card_id	int
userId	user_id	int
battleId	battle_id	int
title	title	char[]
paradigm	paradigm	char[]
description	description	char[]
typing	typing	char[]
difficulty	difficulty	char[]
yearCreated	year_created	int
imageUrl	image_url	char[]

JSON	C	Type
creator	creator_id	int
owner	owner_id	int
status	status	char[]
value	value	int
isAuthenticated	is_authenticated	int
authenticatedBy	authenticated_by	int
authenticatedAt	authenticated_at	long
winCount	win_count	int
isLegitimate	is_legitimate	int
stakeVA	stake_va	int
createdAt	created_at	long

6. Lien avec la blockchain

6.1 Principe

Les structures en mémoire sont dérivées de la blockchain.

Toute modification validée est d'abord écrite dans un bloc, puis répercutée en mémoire.

6.2 Ordre des opérations

1. Le serveur mine un nouveau bloc
2. Le bloc est ajouté à la blockchain
3. Le bloc est sauvegardé en base de données
4. La structure en mémoire est mise à jour

6.3 Exemple concret

```
/* 1. Mine un bloc */
Block block;
block.id = blockchain.size;
block.timestamp = time(NULL);
snprintf(block.data, sizeof(block.data), "ACTION_RECOMMEND|%d|%d", user_id,
card_id);
mine_block(&block);

/* 2. Ajoute a la blockchain */
pthread_mutex_lock(&mutex_blockchain);
add_block(&blockchain, &block);
pthread_mutex_unlock(&mutex_blockchain);
```

```

/* 3. Sauvegarde en BDD */
save_block_to_db(&block);

/* 4. Met a jour la memoire */
pthread_mutex_lock(&mutex_cartes);
Carte *carte = find_carte(card_id);
carte->value = carte->value + 1;
pthread_mutex_unlock(&mutex_cartes);

```

7. Lien avec PostgreSQL

7.1 Principe

Les blocs sont sauvegardés dans la table **blocks**.

Les données courantes sont sauvegardées dans les tables **cards**, **users**, **battles**, **votes**, **legitimacies**.

7.2 Mapping structures C vers tables SQL

Structure C	Table SQL	Colonnes principales
Carte	cards	id, title, paradigm, description, typing, difficulty, year_created, image_url, creator_id, owner_id, status, value, is_authenticated, win_count, created_at
Utilisateur	users	id, username, is_bot, reco_count, legitimacy_count, created_at
Recommandation	recommendations	id, card_id, user_id, active, created_at
Battle	battles	id, card1_id, card2_id, winner_id, status, started_at, ended_at
Vote	votes	id, battle_id, user_id, card_id, weight, signature, voted_at
ActionCarte	legitimacies	id, card_id, user_id, status, requested_at, granted_at
Block	blocks	id, timestamp, data, prev_hash, nonce, hash, created_at

7.3 Fonctions de connexion

```

int db_connect();
void db_disconnect();
int db_save_block(Block *block);
int db_load_blocks(Block **blocks, int *count);
int db_save_card(Carte *card);
int db_update_card_value(int card_id, int new_value);
int db_save_user(Utilisateur *user);
int db_save_recommendation(Recommandation *reco);
int db_save_battle(Battle *battle);
int db_save_vote(Vote *vote);

```

7.4 Structure de la connexion

```
static PGconn *conn = NULL;
```

7.5 Exemple d'insertion en BDD

Exemple pour sauvegarder une carte.

```
int db_save_card(Carte *card) {
    char query[1024];
    snprintf(query, sizeof(query),
        "INSERT INTO cards (title, paradigm, description, typing,
difficulty, "
        "year_created, image_url, creator_id, owner_id, status, value) "
        "VALUES ('%s', '%s', '%s', '%s', '%s', %d, '%s', %d, %d, '%s',
%d)",
        card->title, card->paradigm, card->description,
        card->typing, card->difficulty, card->year_created,
        card->image_url, card->creator_id, card->owner_id,
        card->status, card->value);

    PGresult *res = PQexec(conn, query);
    int status = PQresultStatus(res);
    PQclear(res);

    return (status == PGRES_COMMAND_OK) ? 0 : -1;
}
```

8. Prototypes de fonctions

8.1 Module Carte

```
int create_card(char *title, char *paradigm, char *description,
                char *typing, char *difficulty, int year_created,
                char *image_url, int creator_id);
int recommend_card(int user_id, int card_id);
int repudiate_recommendation(int user_id, int card_id);
Carte* find_carte(int card_id);
int update_card_status(int card_id, char *status);
```

8.2 Module Utilisateur

```
int create_user(char *username, int socket_fd);
Utilisateur* find_utilisateur(int user_id);
```

```
Utilisateur* find_utilisateur_by_username(char *username);  
int remove_utilisateur(int user_id);
```

8.3 Module Battle

```
int launch_battle(int card1_id, int card2_id);  
int vote_battle(int user_id, int battle_id, int card_id);  
int apply_battle_result(int battle_id);  
Battle* find_battle(int battle_id);
```

8.4 Module Legitimation

```
int request_legitimation(int user_id, int card_id);  
int claim_card(int user_id, int card_id);  
int authenticate_card(int user_id, int card_id);
```

8.5 Module Blockchain

```
int blockchain_init(int difficulty);  
int mine_block(Block *block);  
int add_block(Blockchain *bc, Block *block);  
int verify_chain();  
char* compute_hash(Block *block);  
Block* get_last_block();
```

8.6 Module BDD

```
int db_connect();  
void db_disconnect();  
int db_save_block(Block *block);  
int db_load_blocks(Block **blocks, int *count);  
int db_save_card(Carte *card);  
int db_update_card_value(int card_id, int new_value);
```

8.7 Module Protocole

```
char* parse_json_request(char *buffer);  
char* create_json_response(char *status, char *action, char *data);  
char* create_json_error(char *code, char *message);
```

8.8 Module Réseau

```
int network_init(int port);
int network_accept(int server_fd);
void network_close(int fd);
void* client_thread(void *arg);
```

9. Gestion des erreurs

9.1 Principe

Quand une action échoue, le serveur :

1. N'ajoute PAS de bloc à la blockchain
2. N'écrit PAS en base de données
3. Renvoie un message d'erreur au client
4. Garde les structures en mémoire intactes

9.2 Cas d'erreur par action

LOGIN : USERNAME_TAKEN, SERVER_BUSY, INVALID_DATA

CREATE_CARD : INVALID_DATA, DUPLICATE_CARD, INTERNAL_ERROR

RECOMMEND : CARD_NOT_FOUND, CARD_NOT_ACTIVE, ALREADY_RECOMMENDED

REPUDIATE : CARD_NOT_FOUND, NOT_RECOMMENDED

START_BATTLE : CARD_NOT_FOUND, CARD_NOT_ACTIVE, CARDS_NOT_COMPATIBLE, VA_TOO_LOW, BATTLE_ALREADY_ACTIVE

VOTE_BATTLE : BATTLE_NOT_FOUND, ALREADY_VOTED, NOT_ALLOWED

REQUEST_LEGITIMACY : CARD_NOT_FOUND, THRESHOLD_NOT_REACHED

CLAIM_CARD : CARD_NOT_FOUND, NOT_LEGITIMATE

AUTHENTICATE_CARD : CARD_NOT_FOUND, NOT_LEGITIMATE, ALREADY_AUTHENTICATED

9.3 Format des erreurs

```
{
  "status": "ERROR",
  "code": "CARD_NOT_FOUND",
  "message": "Carte introuvable."
}
```

10. Cycle de vie d'une donnée

10.1 Création

L'utilisateur envoie une requête. Le serveur valide. Le serveur mine un bloc. Le serveur sauvegarde. Le serveur met à jour la mémoire.

10.2 Modification

L'utilisateur envoie une requête. Le serveur valide. Le serveur mine un bloc. Le serveur met à jour la BDD. Le serveur met à jour la mémoire.

10.3 Suppression (inactivation)

L'utilisateur envoie une requête. Le serveur valide. Le serveur mine un bloc. Le serveur change le statut en INACTIVE. La donnée reste dans la BDD.

10.4 Restauration

Au démarrage du serveur :

1. Connexion à PostgreSQL
2. Chargement des blocs depuis la table **blocks**
3. Reconstruction de la blockchain en mémoire
4. Reconstruction des listes cartes, utilisateurs, battles

11. Architecture des modules

11.1 Vue d'ensemble

Le serveur est structuré en 3 catégories de modules.

Modules principaux : main.c, network.c, client_handler.c

Modules métier : carte.c, battle.c, legitimatation.c

Modules techniques : blockchain.c, db.c, protocol.c, utils.c

11.2 Diagramme

Le diagramme PlantUML complet est disponible dans :

- [phase1-mai-albine/diagrammes/00-architecture-serveur.puml](#)
- [phase1-mai-albine/diagrammes/00-architecture-serveur.png](#)

11.3 Flux d'une requête

1. main.c accepte une connexion
2. network.c crée un thread pour le client
3. client_handler.c lit la requête
4. protocol.c parse le JSON

5. Le module métier (carte.c, battle.c, legitimisation.c) traite l'action
6. blockchain.c mine un bloc
7. db.c sauvegarde en base
8. protocol.c crée la réponse
9. client_handler.c envoie la réponse

12. Récapitulatif

Structure	Champs	Taille	Utilisation
Carte	15	~1200 octets	Cartes langages
Utilisateur	7	~100 octets	Clients connectés
Recommandation	5	~20 octets	Recommandations
Battle	9	~40 octets	Battles
Vote	5	~20 octets	Votes
ActionCarte	7	~30 octets	Légitimations, revendications, authentications
Block	6	~1200 octets	Blocs blockchain
Blockchain	5	~50 octets	Chaîne complète

13. Validation

Rôle	Nom	Statut
Serveur C	Mai	En attente
Serveur C	Albine	En attente
Blockchain + BDD	Iyore	En attente
